Descripción del artefacto de código fuente

EduAirControl

Jhon Jairo Cortez Vargas

María de los Ángeles Olaya García

Servicio Nacional de Aprendizaje (SENA) – Centro de Formación SENA Industria

Tecnólogo en Análisis y Desarrollo de Software — Ficha 3145556

Karol Daniela Correa

Neiva, Huila

2026

Resumen

Este documento describe el artefacto de código fuente del proyecto EduAirControl, una plataforma orientada a la gestión, monitoreo y control de ambientes educativos mediante Internet de las Cosas (IoT). El artefacto integra un backend distribuido en microservicios, una aplicación web, una aplicación móvil, contratos de API, scripts de base de datos e infraestructura de ejecución. La copia anexada fue organizada a partir del ZIP entregado por el equipo del proyecto y excluye dependencias generadas, repositorios Git internos, archivos de configuración local y claves privadas de desarrollo. El firmware del ESP32 no fue encontrado en el ZIP recibido; por ello se documenta como componente pendiente de incorporación.

Palabras clave: código fuente, EduAirControl, microservicios, IoT, React, Spring Boot, React Native.

Descripción del artefacto

En el contexto del proyecto, el artefacto de código fuente corresponde al conjunto de archivos técnicos que permiten construir, ejecutar, probar y desplegar EduAirControl. No se trata de un único archivo, sino de una composición de módulos que trabajan conjuntamente para recibir, procesar, almacenar y visualizar información ambiental de aulas y otros espacios educativos.

La organización presentada en este anexo permite relacionar los productos de código con la arquitectura documentada, los contratos de integración y los procedimientos de implementación. El código conserva sus extensiones y convenciones originales, ya que las normas APA se aplican a la documentación académica que explica el artefacto, no a la sintaxis de los lenguajes de programación.

Contenido de la entrega

Tabla 1

Componente	Contenido	Tecnologías principales
Backend	API Gateway y siete microservicios de dominio.	Java, Spring Boot, Spring Cloud Gateway, Maven
Frontend web	Interfaz para consulta y administración desde navegador.	React, Vite, React Router, Recharts
Aplicación móvil	Cliente móvil para monitoreo, administración y aprovisionamiento BLE.	Expo, React Native, React Navigation, BLE
Contratos API	Definiciones de rutas, esquemas y respuestas de integración.	OpenAPI/YAML
Infraestructura	Contenedores, composición local, Kubernetes, observabilidad y scripts.	Docker Compose, Kubernetes, Prometheus, Grafana
Base de datos	Migraciones y estructuras SQL organizadas por microservicio.	PostgreSQL, Liquibase, SQL
Firmware IoT	Nota de componente pendiente; no había archivos de firmware en el ZIP.	ESP32, DHT11, KY-038, MQ-135

Nota. Elaboración propia con base en el código fuente recibido y la arquitectura del proyecto.

Estructura de carpetas

La carpeta anexada en 14_Anexos/Codigo_Fuente contiene la siguiente organización:

01_Backend_Microservicios: API Gateway y servicios de seguridad, aulas, sensores, monitoreo, usuarios y experiencia.

02_Frontend_Web: aplicación web React/Vite.

03_Aplicacion_Movil: aplicación Expo/React Native.

04_Contratos_API: contratos OpenAPI del sistema.

05_Infraestructura: Docker Compose, manifiestos Kubernetes, observabilidad y scripts.

06_Base_Datos: scripts SQL y migraciones extraídas del backend.

07_Firmware_IoT: documentación de ausencia del firmware y requisitos de incorporación.

README.md: instrucciones de uso y procedencia.

MANIFEST.txt: inventario de archivos incluido en el paquete.

Backend y servicios

El backend se distribuye en servicios independientes. El API Gateway centraliza el acceso y aplica validaciones de seguridad, identificación de correlación y limitación de solicitudes. Los servicios de dominio separan las responsabilidades de identidad, administración de ambientes educativos, gestión de sensores, monitoreo ambiental, perfiles y preferencias de usuario. Esta separación es coherente con una arquitectura basada en microservicios y facilita el despliegue y la evolución independiente de cada componente.

Tabla 2

Servicio	Responsabilidad	Puerto documentado
API Gateway	Entrada única y políticas transversales.	8080
MS Security	Autenticación, autorización, usuarios institucionales y JWT.	8081
MS Classroom Management	Campus y ambientes educativos.	3002
MS Environment Monitoring	Mediciones, análisis, umbrales y alertas.	3003
MS Sensor Management	Catálogo, instalación y flujo de sensores.	3004

MS User Management	Perfil del usuario.	3007
MS User Experience	Preferencias, favoritos, búsquedas y notificaciones.	Según configuración del servicio

Nota. Elaboración propia con base en el código fuente recibido y la arquitectura del proyecto.

Clientes web y móvil

La aplicación web ofrece la interfaz de consulta y administración desde un navegador. El frontend móvil permite acceder a las funciones de monitoreo desde un dispositivo y contiene componentes relacionados con el aprovisionamiento Bluetooth Low Energy del ESP32. Ambos clientes consumen los contratos del API y dependen de las variables de entorno definidas para cada ambiente.

Base de datos e integración

Los scripts SQL se mantienen asociados a los microservicios y se consolidaron en la carpeta 06_Base_Datos únicamente para facilitar la entrega y revisión académica. La infraestructura descrita utiliza PostgreSQL para persistencia, Redis para estado o caché y RabbitMQ para mensajería entre servicios. Las migraciones y tablas deben ejecutarse mediante el procedimiento definido en el documento de implementación, sin sustituir los archivos originales del backend.

Infraestructura y ejecución

La carpeta de infraestructura contiene los archivos necesarios para levantar el entorno local con Docker Compose, esperar la disponibilidad de los servicios y ejecutar el smoke test. También incluye manifiestos Kubernetes y archivos de observabilidad. Los archivos de variables locales y secretos no se entregan con valores reales; deben ser creados por el responsable de la instalación a partir de las plantillas proporcionadas.

Firmware y hardware IoT

El ZIP revisado no contiene firmware del ESP32 en formato .ino, .cpp o .h. En consecuencia, el artefacto entregado documenta la ausencia, pero no afirma que el firmware esté incluido. Para completar este componente se debe añadir el programa del ESP32, el

diagrama de conexión, la versión del framework utilizado, la asignación de pines, el formato de la carga útil y las evidencias de lectura del DHT11, KY-038 y MQ-135.

Criterios de reproducción

La reproducción del artefacto requiere clonar o copiar la carpeta, instalar las dependencias del frontend web y móvil, configurar las variables de entorno desde sus ejemplos, levantar la infraestructura y ejecutar las pruebas indicadas en el Plan de Pruebas. La instalación debe realizarse en un entorno separado del repositorio fuente y nunca debe reemplazar claves de producción ni datos reales.

Backend e infraestructura: ejecutar Docker Compose desde 05_Infraestructura.

Frontend web: ejecutar npm install y npm run dev en 02_Frontend_Web.

Aplicación móvil: ejecutar npm install y npm start desde la carpeta de Expo.

Pruebas Java: ejecutar Maven con JDK 17 en cada microservicio.

Firmware: agregar el código faltante antes de declarar completa la cadena IoT.

Control de seguridad y exclusiones

Para esta entrega se excluyeron las carpetas .git, node_modules, target, .vite, .expo y dist, porque son metadatos, dependencias o artefactos generados. También se excluyeron los archivos .env y las claves de desarrollo dev-private.pem y dev-public.pem encontrados en el material original. Esta decisión evita divulgar secretos y mantiene el paquete enfocado en el código fuente reproducible.

Procedencia y repositorios

El material fue organizado a partir del ZIP del proyecto entregado para revisión. Los repositorios declarados por el equipo son: EduAirControl para el código web y backend, y EduAirControlMOBILE para la aplicación móvil. Los enlaces se conservan también en el README del artefacto.

Referencias

Docker. (s. f.). Docker Compose documentation. https://docs.docker.com/compose/

Espressif Systems. (s. f.). ESP32 documentation. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/

Expo. (s. f.). Expo documentation. https://docs.expo.dev/

Kubernetes Authors. (s. f.). Kubernetes documentation. https://kubernetes.io/docs/

OpenAPI Initiative. (s. f.). OpenAPI specification. https://spec.openapis.org/oas/latest.html

React Team. (s. f.). React documentation. https://react.dev/

Spring. (s. f.). Spring Boot documentation. https://docs.spring.io/spring-boot/index.html